Institut National des Postes et Télécommunications (INPT), Rabat
Filière Smart ICT – 3e année

IA agentique pour le RIC en O-RAN

Proposition d'implémentation – version révisée

Réalisé par :
Souhail Bourhim
Ghita Boulmani

Encadré par :
Pr. Mustapha Benjillali

Octobre 2026

Résumé. Ce document présente ce que nous proposons de réaliser concrètement dans le cadre du projet, au-delà de l'étude bibliographique. L'idée est de construire, sur une plateforme 5G open source, un contrôleur dans lequel un agent basé sur un LLM traduit l'objectif de l'opérateur en consignes pour un contrôleur par apprentissage par renforcement (RL), avec une couche de vérification entre les deux. Nous comparerons ensuite cette approche à des solutions plus simples sur les mêmes scénarios.

Le projet s'appuie sur les trois articles de référence : l'architecture hiérarchique « le LLM guide, le RL exécute » de Bao et al. (LLM-hRIC), le problème d'allocation de ressources entre slices étudié par Wu et al., et l'approche multi-agents DRL de Rezazadeh et al., qui servira de référence côté RL. L'ensemble tourne sur une seule machine Linux, sans matériel radio.

Cette version ajoute Hermes Agent, un agent open source existant, comme simple *interface opérateur*. Hermes transmet les intentions à une passerelle qui n'expose que trois outils (`submit_intent`, `get_kpis`, `get_status`). Il ne remplace ni LangGraph, ni le vérificateur déterministe, ni la xApp RL ; il n'a aucun accès aux commandes E2 ni au partage des PRB, et il reste hors de la boucle temps réel. Ce n'est pas une intégration O-RAN native. Les hypothèses ajoutées sont repérées dans le texte par **[H1]** à **[H8]** et listées en section 10.

1. Contexte et problématique

Une cellule 5G dispose d'une capacité radio limitée, mesurée en blocs de ressources physiques (PRB). Cette capacité est partagée entre plusieurs *slices*, chacun ayant des besoins différents (figure 1). La demande varie au cours de la journée, donc la répartition des PRB entre slices doit être ajustée en continu.

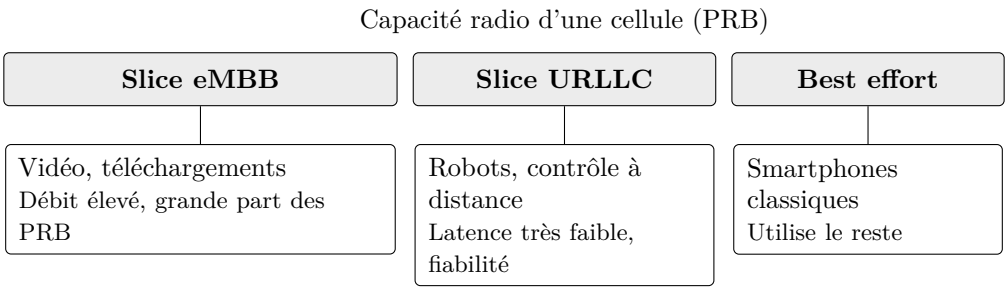


Fig. 1 : Une cellule, trois slices aux besoins différents, une répartition à ajuster en permanence.

Dans l'architecture O-RAN, cette décision est prise dans le RAN Intelligent Controller (RIC). Le RIC héberge des applications de contrôle : les xApps dans le Near-RT RIC (boucles de 10 ms

à 1 s) et les rApps dans le Non-RT RIC (au-delà d'une seconde).

Côté opérateur, l'objectif est de pouvoir exprimer une intention en langage naturel, par exemple « garantir 10 Mb/s au slice URLLC entre 18h et 19h », et de laisser le réseau s'en charger. La question à laquelle nous voulons répondre est donc la suivante :

> *Comment permettre à un opérateur de piloter le RAN en langage naturel, tout en gardant des décisions rapides et en empêchant l'IA de prendre des actions dangereuses ?*

2. Limites des approches existantes

Approche	Principe	Avantage	Limite
Règles fixes	Répartition statique, modifiée manuellement	Simple, prévisible	Trop lente et rigide face à une demande variable
RL seul	Une politique entraînée associe les KPI à une répartition des PRB	Rapide (ms), efficace en optimisation	Ne connaît que l'objectif appris ; un nouvel objectif impose un nouvel entraînement
LLM seul	Le LLM lit l'objectif et fixe lui-même la répartition	Comprend le langage naturel, explique ses choix	Trop lent pour les boucles rapides (∼1 s ou plus) ; peut proposer des actions erronées

Tab. 1 : Comparaison des approches existantes.

Le terme « agent » a deux sens dans les articles étudiés. Un agent RL est une politique entraînée qui associe un état à une action, sans raisonnement. Un système d'IA agentique est un LLM qui planifie, appelle des outils, observe le résultat et se corrige. Notre proposition combine les deux.

3. Solution proposée

Nous séparons les rôles selon l'échelle de temps et selon le niveau de confiance. L'opérateur s'exprime dans Hermes Agent, qui sert d'interface (section 5). Le service LangGraph, placé dans le Non-RT RIC, interprète l'intention et propose un objectif. Un vérificateur déterministe contrôle les limites et la sécurité de cet objectif avant toute modification de l'objectif du RL. La xApp RL, dans le Near-RT RIC, reste seule responsable des décisions rapides d'allocation des PRB. Hermes et les LLM restent hors de la boucle temps réel (figure 2).

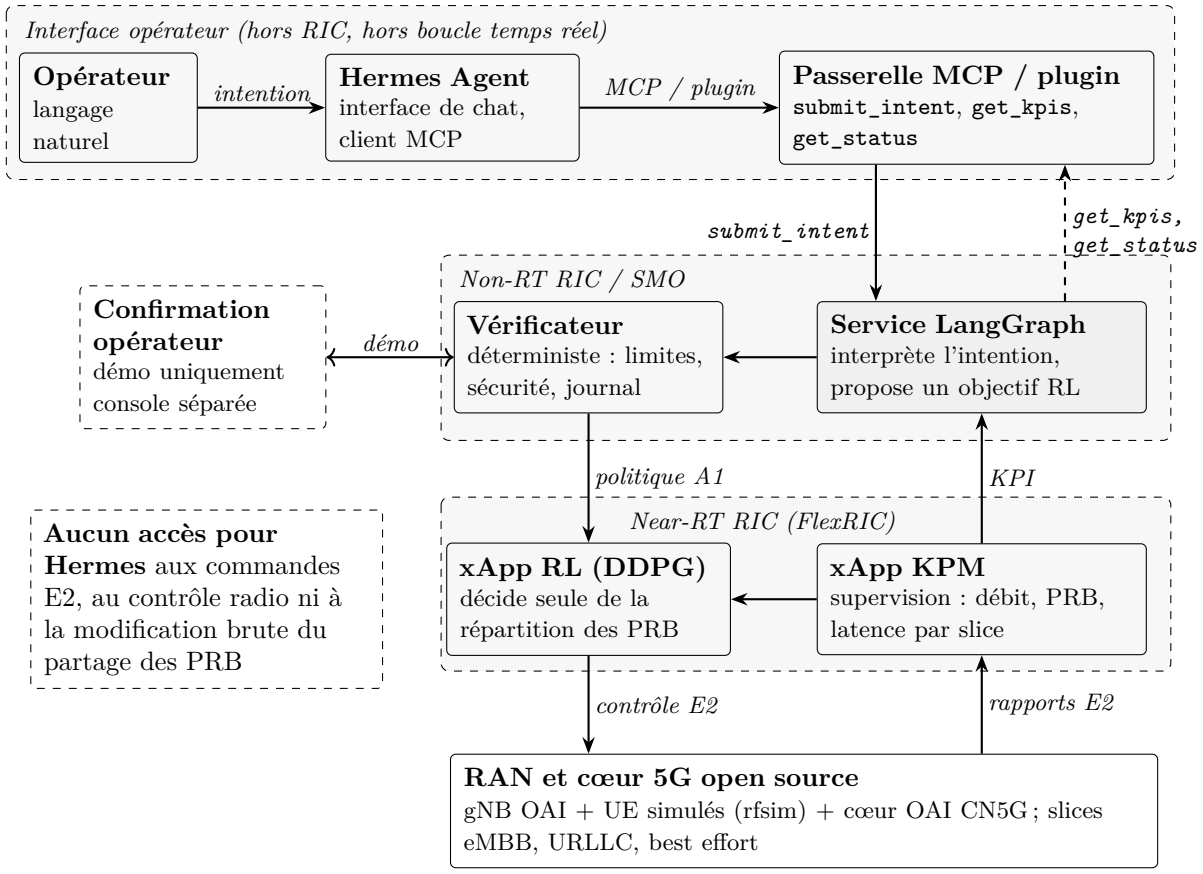


Fig. 2 : Architecture révisée. Hermes Agent est une couche d'interface sans accès au RIC ; LangGraph, le vérificateur déterministe et la xApp RL gardent leur rôle. Traits pleins : demandes et actions ; trait discontinu : lecture seule.

Notre contribution porte sur cinq points :

1. un contrôle hiérarchique LLM + RL, inspiré de Bao et al., appliqué au problème d'allocation entre slices de Wu et al. ; le LLM n'intervient jamais dans la boucle rapide ;
2. une couche de sécurité déterministe qui vérifie chaque objectif proposé par le LLM et compte les actions bloquées ;
3. une mise en œuvre sur une vraie pile open source (OpenAirInterface + FlexRIC), alors que la plupart des travaux du domaine restent en simulation ;
4. une interface opérateur volontairement étroite (Hermes Agent derrière une passerelle à trois outils), évaluée séparément du contrôle ;
5. une évaluation comparative de quatre contrôleurs sur les mêmes scénarios (section 7).

4. Fonctionnement du service LangGraph

Nous utilisons un seul agent, développé avec LangGraph et exposé comme un petit service HTTP. Il suit une boucle planification – action – observation (approche ReAct), avec une vérification systématique avant chaque action (figure 3).

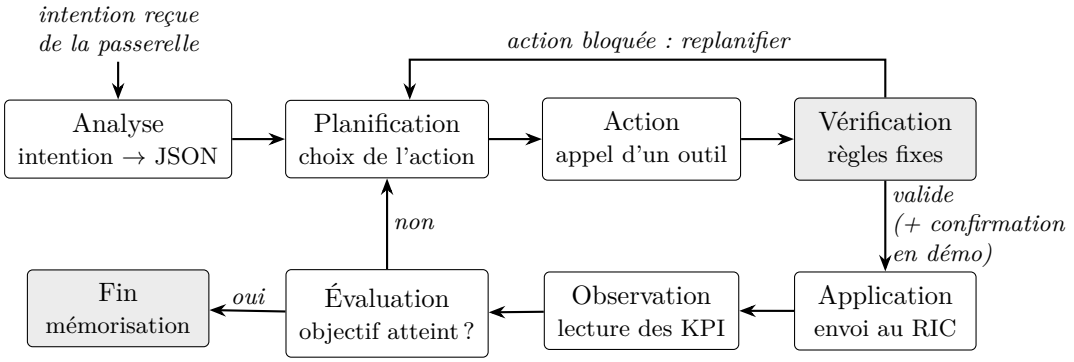


Fig. 3 : Boucle de décision du service LangGraph.

Le service n'agit jamais directement sur le réseau : il ne peut qu'appeler les outils internes ci-dessous, et tout appel qui modifie la configuration passe par le vérificateur. Ces outils internes ne sont pas ceux que voit Hermes (section 5).

Outil interne	Rôle
`get_kpis(slice, fenetre)`	Lit le débit, l'usage des PRB et la latence récents
`set_rl_objective(poids, contraintes)`	Modifie l'objectif optimisé par la xApp RL (levier principal)
`set_slice_share(parts)`	Fixe directement la répartition des PRB. Désactivé dans le contrôleur proposé ; utilisé seulement pour le contrôleur « LLM seul » de l'évaluation, toujours derrière le vérificateur[H6]
`wait_and_observe(secondes)`	Laisse le réseau se stabiliser avant la vérification suivante

Tab. 2 : Outils internes du service LangGraph.

Pour le modèle, nous prévoyons un LLM local via Ollama (un modèle ouvert d'environ 8 milliards de paramètres avec appel d'outils, par exemple Qwen). Il est gratuit, fonctionne hors ligne et reste réaliste pour un déploiement en périphérie du réseau. Nous testerons aussi un modèle plus grand accessible par API (Claude ou GPT) sur les mêmes scénarios, afin de mesurer le gain en précision, en latence et en coût.

Certains travaux récents (MX-AI, AgentRAN) utilisent plusieurs agents. Pour un projet d'un semestre à deux, nous préférons un seul agent, plus simple à mettre au point. Si le temps le permet, nous ajouterons un agent « analyste » en lecture seule, chargé d'expliquer les anomalies observées dans les KPI.

5. Interface opérateur : Hermes Agent

Pour que l'opérateur formule ses intentions dans une interface de conversation existante, nous plaçons Hermes Agent en amont du service LangGraph. Hermes Agent est un agent open source (licence MIT) développé par Nous Research, qui se connecte à des outils externes par le protocole MCP ou par des plugins[H1]. Ce n'est pas un composant O-RAN et il n'a aucun support natif du RIC : du point de vue du réseau, c'est un client MCP ordinaire. Il ne remplace ni LangGraph, ni le vérificateur déterministe, ni la xApp RL, et le rôle du contrôleur RL ne change pas.

5.1 Chaîne de traitement

1. L'opérateur formule une intention dans Hermes.
2. Hermes la transmet à la passerelle (serveur MCP ou plugin Hermes personnalisé) avec l'outil `submit_intent`.
3. La passerelle transmet l'intention au service LangGraph, qui l'interprète et propose un objectif[H5].
4. Le vérificateur déterministe contrôle les limites et la sécurité de l'objectif proposé. Un objectif rejeté n'atteint jamais la xApp RL.
5. Dans le scénario de démonstration, l'opérateur confirme dans une console séparée avant l'application[H4].
6. L'objectif validé est transmis à la xApp RL par une politique A1. La xApp RL reste seule responsable des décisions rapides d'allocation des PRB dans le Near-RT RIC.
7. L'opérateur suit l'avancement dans Hermes avec `get_status` et `get_kpis`.

Hermes et les LLM ne font donc jamais partie de la boucle temps réel.

5.2 Outils exposés par la passerelle

La passerelle expose exactement trois outils, et aucun autre.

Outil	Rôle	Effet sur le réseau
`submit_intent(texte)`	Transmet au service LangGraph le texte de l'intention, tel qu'écrit par l'opérateur[H2] ; renvoie un identifiant et l'état initial	Crée une demande. Rien n'est appliqué sans validation du vérificateur (et confirmation en démo)
`get_kpis(slice, fenetre)`	Renvoie les KPI récents (débit, PRB, latence)	Lecture seule
`get_status(id)`	Renvoie l'état de l'intention (reçue, interprétée, rejetée avec motif, en attente de confirmation, appliquée, expirée) et l'objectif RL en vigueur	Lecture seule

Tab. 3 : Outils exposés à Hermes par la passerelle.

5.3 Ce que Hermes ne peut pas faire

- Hermes ne reçoit **aucun accès direct** aux commandes E2, au contrôle radio ni à la modification brute du partage des PRB.
- Aucun outil exposé ne modifie la configuration : `submit_intent` ne fait que soumettre une demande.
- Toute action qui modifie la configuration passe par le service de vérification.
- La confirmation opérateur ne passe pas par Hermes : sinon le LLM pourrait se l'accorder lui-même[H4].

La configuration MCP de Hermes permet de n'enregistrer, pour chaque serveur, qu'une liste blanche d'outils (clé `include`) ; nous y déclarons les trois outils de la passerelle. Ces garanties

ne reposent pas seulement sur la liste d'outils. D'après sa documentation, Hermes est un agent généraliste qui peut aussi disposer d'outils comme un terminal, des fichiers ou le web[H3]. Pour ce projet, nous désactivons ces outils et nous exécutons Hermes dans un conteneur dédié dont le seul accès réseau est le port de la passerelle. La passerelle n'a elle-même qu'un interlocuteur, le service LangGraph, sans route vers FlexRIC ni vers OAI[H5].

5.4 Prototype minimal

Le prototype montre le circuit complet avec un périmètre réduit. Il est d'abord réalisé contre le simulateur (le service LangGraph s'appuie alors sur l'environnement gym), puis contre la plateforme OAI + FlexRIC si elle est prête[H8].

1. Un serveur MCP en Python implémente les trois outils et appelle l'API HTTP du service LangGraph.
2. Hermes est configuré avec ce serveur comme seule source d'outils.
3. Scénario de démonstration : l'opérateur écrit « garantir 10 Mb/s au slice URLLC pendant 10 minutes ». Hermes appelle `submit_intent` ; la passerelle renvoie l'état « en attente de confirmation » ; l'opérateur confirme dans la console séparée ; l'objectif est appliqué ; l'opérateur demande où en est le réseau et Hermes affiche l'état et les KPI par slice avec `get_status` et `get_kpis`.
4. Cas de rejet : une intention hors limites (par exemple un débit irréaliste) est refusée par le vérificateur, et le motif apparaît dans `get_status`.
5. Test négatif : une demande hors périmètre (« fixe le partage des PRB à 90/5/5 ») ne doit produire aucun changement d'état, car aucun outil ne le permet.

Le prototype est réussi si ces trois cas se déroulent comme prévu et si le journal de la passerelle ne contient que les trois outils exposés.

6. Plateforme expérimentale

La plateforme tourne sur un ordinateur portable (Ryzen 9, 16 cœurs, 32 Go de RAM, GPU RTX 5070 de 8 Go) sous Ubuntu dans WSL2, avec Docker. Le lien radio est simulé, aucun matériel SDR n'est nécessaire. Le module SCTP, nécessaire aux interfaces du cœur OAI, se charge correctement dans WSL2.

Composant	Outil	Réutilisé / développé
Station de base et UE	gNB et nrUE OpenAirInterface, mode rfsim, 2 à 3 UE	Réutilisé
Cœur 5G	OAI CN5G avec 2 à 3 slices	Réutilisé
Near-RT RIC	FlexRIC	Réutilisé
Trafic	iperf3 sur chaque UE, charge variable	Scénarios à écrire
xApp de supervision	Modèle de service KPM de FlexRIC + stockage des mesures	Développé
xApp de contrôle	Modèle de service de contrôle des slices de FlexRIC	Développé
Politique RL	DDPG, entraîné dans un simulateur Python (gym) puis déployé	Développé
Service LangGraph et vérificateur	LangGraph, Ollama, API HTTP	Développé
Interface opérateur	Hermes Agent, dans un conteneur isolé, outils généralistes désactivés[H1] [H3]	Réutilisé, configuré
Passerelle	Serveur MCP en Python (ou plugin Hermes), trois outils	Développé
Console de confirmation	Ligne de commande ou petite page web du service de vérification[H4]	Développé (minimal)

Tab. 4 : Composants de la plateforme.

Nous associons FlexRIC à OAI plutôt qu'à srsRAN, car le contrôle des slices dans FlexRIC est mieux supporté avec OAI ; c'est aussi la combinaison utilisée dans MX-AI. L'entraînement du RL se fait d'abord en simulation, car il serait trop lent sur la plateforme émulée.

7. Méthode d'évaluation

Nous menons deux évaluations séparées. La première mesure le contrôle du réseau ; la seconde mesure l'interface. Ainsi, la performance de l'interface n'est pas confondue avec celle du contrôle RL, ni l'inverse.

7.1 Évaluation du contrôle : quatre contrôleurs

Les mêmes scénarios de trafic sont exécutés avec quatre contrôleurs. Hermes n'intervient dans aucun d'eux : les intentions sont injectées directement dans le service LangGraph, et seules les intentions correctement extraites (vérifiées au préalable) sont retenues pour cette comparaison[H7].

Contrôleur	Ce qu'il représente
Répartition statique égale	Référence sans intelligence
RL seul	Approche de type Rezazadeh et al. / Wu et al.
LLM seul, action directe	Approche agentique naïve
LLM (objectifs) + RL (exécution)	**Notre proposition**, d'après Bao et al.

Tab. 5 : Contrôleurs comparés.

Scénarios, tous générés avec iperf3 pour que chaque contrôleur voie le même trafic :

1. charge stable, puis pic soudain sur un slice ;
2. nouvelle intention en cours d'exécution (« protéger URLLC pendant 10 minutes ») ;
3. deux intentions contradictoires.

Métriques :

- proportion du temps où l'intention de chaque slice est respectée ;
- débit et latence par slice ;
- temps de réaction après un pic de charge ou une nouvelle intention ;
- nombre d'actions dangereuses proposées puis bloquées par le vérificateur ;
- latence et coût du LLM du service LangGraph (modèle local vs modèle distant).

7.2 Évaluation de l'interface : Hermes et passerelle

Cette évaluation s'arrête à la décision du vérificateur : elle n'utilise ni le débit ni la latence radio, et ses résultats ne servent pas à classer les contrôleurs. Elle reprend le scénario des intentions ambiguës ou mal formulées, auparavant dans la comparaison des contrôleurs[**H7**].

Mesure	Définition	Source des données
Latence de l'interface	Temps entre l'envoi du message de l'opérateur et l'arrivée de `submit_intent` à la passerelle, puis entre la réponse de la passerelle et son affichage ; médiane et 95e centile sur des répétitions. Le temps de décision du service LangGraph est exclu	Horodatages de la passerelle et journal de Hermes
Fidélité de transmission	Part des intentions arrivées à `submit_intent` sans reformulation par le LLM de Hermes[**H2**]	Texte saisi comparé au texte reçu par la passerelle
Qualité d'extraction de l'intention	Sur un jeu d'intentions annotées à la main[**H8**] : exactitude par champ (slice, métrique, valeur cible, fenêtre de temps), part des intentions entièrement correctes, détection des cas ambigus	Sortie structurée du service LangGraph comparée aux annotations
Appels d'outils	Par outil : appels réussis, rejetés par la passerelle (arguments invalides), rejetés par le vérificateur (hors limites), en erreur	Journaux de la passerelle et du vérificateur
Tentatives hors périmètre	Demandes d'une action non exposée (par exemple fixer directement le partage des PRB) : aucun changement d'état attendu	Journal et état du RIC avant et après

Tab. 6 : Mesures de l'interface, indépendantes du contrôle RL.

8. Plan de travail

Le travail est organisé en cinq phases et deux axes menés en parallèle, avec un point de validation à la fin de chaque phase (figure 4). L'axe B n'attend pas la plateforme : le simulateur permet de commencer le RL et l'agent dès la première phase. La phase 4 (interface) ne démarre que lorsque le service LangGraph et son vérificateur fonctionnent ; elle peut se réduire à un client en ligne de commande si le temps manque (section 9). La répartition des axes entre nous deux reste à fixer.

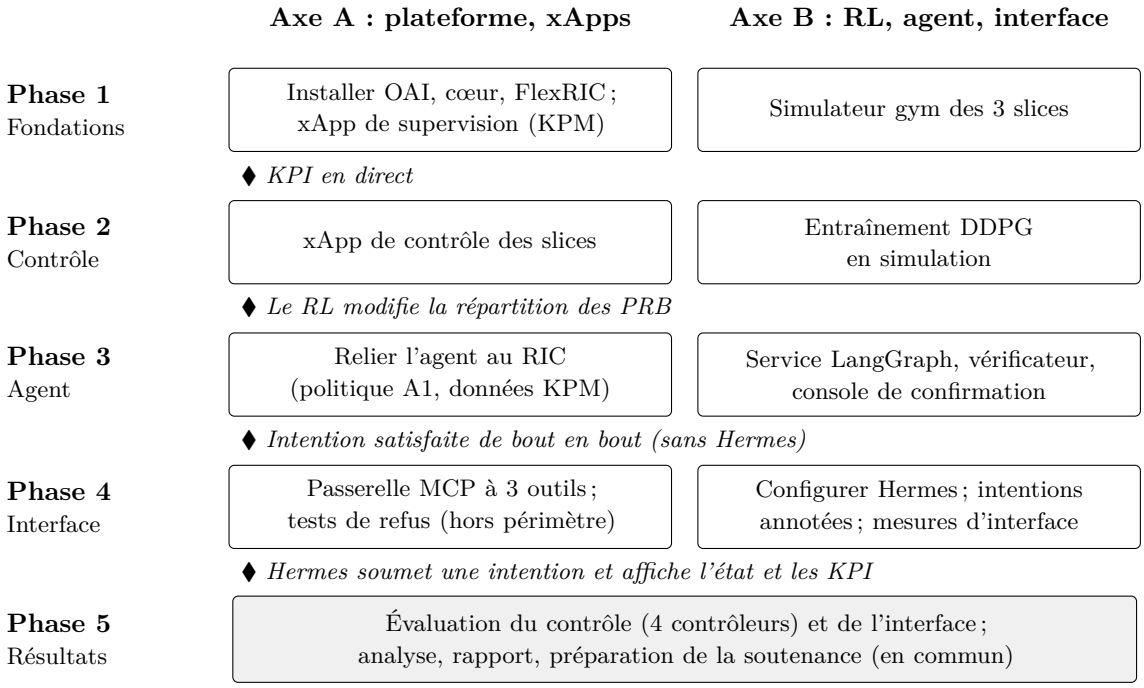


Fig. 4 : Phases du projet. Les losanges marquent les points de validation.

9. Risques et solutions de repli

Risque	Solution de repli
Le contrôle des slices OAI + FlexRIC ne fonctionne pas de façon fiable	Garder la supervision KPM sur la plateforme réelle et faire tourner la boucle de contrôle en simulation (ns-O-RAN ou notre simulateur)
Machine trop limitée pour la plateforme et un LLM local	Utiliser le LLM via API ou sur une seconde machine
La politique RL entraînée en simulation se comporte mal sur la plateforme	Court ré-entraînement sur la plateforme, ou exécutant à règles simples sous le LLM
LLM trop lent ou peu fiable dans l'appel d'outils	Prompts plus courts, schéma JSON imposé, ou modèle plus grand
Hermes difficile à installer ou instable (version, environnement)	La même passerelle est pilotée par un petit client en ligne de commande ; le contrôle et son évaluation n'en dépendent pas
Le LLM de Hermes appelle mal les outils ou reformule l'intention	Texte transmis tel quel, validation de schéma à la passerelle, modèle plus grand ; mesuré dans l'évaluation de l'interface
Hermes contourne la passerelle avec ses outils généralistes	Outils désactivés, conteneur isolé sans route vers FlexRIC ni OAI, tests négatifs du prototype
Dérive du périmètre (mémoire, compétences, messageries de Hermes)	N'utiliser que le chat et MCP ; limiter la phase 4 dans le temps ; abandon possible au profit du client en ligne de commande
Deux LLM en chaîne ajoutent de la latence et des erreurs	Interface hors boucle temps réel ; latence mesurée à part ; bascule vers le client direct si nécessaire

Tab. 7 : Principaux risques identifiés.

10. Hypothèses ajoutées dans cette version

Les points suivants ont été ajoutés ou choisis pour intégrer Hermes Agent. Ils ne viennent pas de la version précédente et restent à valider avec l'encadrant ou par des essais.

H1 Hermes Agent (Nous Research, open source, licence MIT) sert uniquement d'interface de conversation et de client MCP. D'après sa documentation, il prend en charge les serveurs MCP et les plugins ; nous n'avons pas encore testé son installation sur notre machine (WSL2) et la version à utiliser reste à figer.

H2 Le modèle utilisé par Hermes est configurable ; nous supposons qu'il peut s'agir du modèle local (Ollama) ou d'un modèle distant, ce qui reste à vérifier. En local, Hermes et le service LangGraph partageraient le même modèle servi par Ollama, car le GPU (8 Go, dont une partie utilisée par Windows) ne peut pas en charger deux. Hermes transmet le texte de l'opérateur tel quel à `submit_intent` ; l'extraction structurée est faite par LangGraph, ce qui permet de la mesurer à part.

H3 Hermes est isolé : conteneur dédié, outils généralistes désactivés, accès réseau limité à la passerelle. La possibilité de désactiver ces outils par configuration est à confirmer dans la version utilisée.

H4 La confirmation opérateur se fait dans une console séparée du service de vérification (ligne de commande ou petite page web), pas par un outil Hermes. Une demande non confirmée

expire après un délai configurable (valeur à fixer). Les exécutions d'évaluation automatisées n'utilisent pas la confirmation ; seule la démonstration l'exige.

H5 La passerelle n'a qu'un interlocuteur, l'API HTTP du service LangGraph ; `get_kpis` et `get_status` y sont servis en lecture seule (les KPI viennent du stockage de la xApp de supervision).

H6 L'outil interne `set_slice_share` n'est plus un moyen de secours du contrôleur proposé : il est désactivé pour celui-ci et sert seulement au contrôleur « LLM seul », derrière le vérificateur. Il n'est jamais exposé à Hermes.

H7 Pour comparer les quatre contrôleurs, les intentions sont injectées sans Hermes et seules les intentions correctement extraites sont retenues. Le scénario des intentions ambiguës passe dans l'évaluation de l'interface.

H8 Le jeu d'évaluation de l'extraction compte de l'ordre de 40 intentions annotées à la main (français et anglais, dont des cas ambigus) ; ce dimensionnement est une estimation à ajuster. Les mesures d'interface se font d'abord sur le simulateur, puis sur la plateforme si elle est prête.

11. Points à discuter avec l'encadrant

- Disposons-nous d'un serveur ou d'un GPU au laboratoire, ou faut-il prévoir un PC personnel ?
- Un budget API est-il envisageable pour la comparaison avec un modèle distant ?
- Le périmètre de l'évaluation (quatre contrôleurs, trois scénarios, plus l'évaluation de l'interface) vous paraît-il adapté ?
- Hermes Agent est-il acceptable comme composant d'interface dans le périmètre du projet, ou préférez-vous un client plus simple ?
- Quelles sont les échéances à respecter (point d'avancement, rapport, soutenance) ?

Références

[1] Rezazadeh et al., apprentissage par renforcement profond multi-agents pour le slicing du RAN.

[2] Wu et al., IA agentique pilotée par LLM pour la résilience en O-RAN.

[3] Bao et al., LLM-hRIC : contrôle hiérarchique LLM + RL dans le RIC.

[4] MX-AI, agents LLM sur OAI + FlexRIC, arXiv :2508.09197.

[5] AgentRAN, hiérarchie d'agents auto-organisée pour l'Open RAN, arXiv :2508.17778.

[6] xTRUCE, arbitrage sûr entre xApps pilotées par LLM, arXiv :2608.28532.

[7] OpenAirInterface (`https://openairinterface.org`), FlexRIC (`https://gitlab.eurecom.fr/mosaic5g/flexric`), LangGraph, Ollama.

[8] Hermes Agent, Nous Research, open source, licence MIT (`https://hermes-agent.nousresearch.com` ; dépôt `https://github.com/NousResearch/hermes-agent` ; configuration MCP : `https://hermes-agent.nousresearch.com/docs/user-guide/features/mcp`).

[9] Model Context Protocol (MCP) (`https://modelcontextprotocol.io`).